

Problem Statement: AI-Powered “InsightFlow” Dashboard

You are hired as a front-end engineer to build a rich, interactive web application that showcases a seamless blend of modern web design and artificial intelligence. The backend AI services (text generation, sentiment analysis, image classification, etc.) are provided as mocked Python functions that return realistic but static results – your job is to **design and implement the entire user interface** using **Streamlit** and **Python only**. The final product must look professional, feel responsive, and guide users through AI workflows in an intuitive way.

1. Application Overview

Build a multi-page Streamlit application called **InsightFlow** that offers three main AI-powered modules:

Module	Core AI Feature
Smart Inbox	Analyse and summarise customer emails, suggest replies, and detect sentiment/urgency.
Creative Copilot	A chat-style interface for generating marketing copy, blog outlines, or social media posts with live streaming and tone controls.
Vision Board	Upload images and receive AI-generated descriptions, object tags, and a confidence heatmap (simulated).

All AI responses are obtained from placeholder functions (provided in a separate file `ai_stubs.py`). **You are not required to implement any real AI logic.** Your entire effort goes into designing the front-end experience that makes these AI capabilities shine.

2. Technical Requirements

1. Streamlit-only implementation

- Use Streamlit's native widgets, columns, containers, and theming – no external HTML/JavaScript frameworks (except for minimal inline SVG or Markdown).
- All state must be managed with `st.session_state`.

2. Multi-page layout

- Create a polished sidebar navigation that switches between the three modules and an "About" page.
- The sidebar must also contain global settings (e.g., theme toggle light/dark, API key input field – purely cosmetic, as no real calls are made).

3. Advanced UI patterns

- Streaming of AI responses (simulate a word-by-word typing effect for generated text).
- Skeleton loaders and progress bars while "waiting" for AI results.
- Toast-style notifications for success/error/warning states.
- Expandable panels that reveal model metadata (e.g., latency, token count, confidence scores).
- Drag-and-drop file upload zone with image preview (at least 3 file types).

4. Data visualisation

- For the **Smart Inbox**, display a dynamic bar chart of sentiment distribution across all analysed emails.
- For the **Vision Board**, show a simulated heatmap overlay on the uploaded image using a transparent matplotlib figure layered on the original image.
- All charts must be interactive (zoom, hover tooltips).

5. Responsiveness & Accessibility

- UI must adapt gracefully to different viewport widths (use `st.columns` with ratios, flexible containers).
 - Ensure sufficient colour contrast and that interactive elements are keyboard-navigable.
-

3. Detailed Module Specifications

A. Smart Inbox

- The user can **paste a raw email text** or **upload a .txt/.eml file**.
- After submission, the interface shows:
 - Extracted **subject, sender, date** (simulated parsing).
 - A **summary** generated by AI (streaming text).
 - **Sentiment** (positive/neutral/negative) with a semi-circular gauge chart.
 - **Urgency level** (low/medium/high) shown as a colour-coded progress bar.
 - **Suggested reply** that the user can edit in a text area, then copy to clipboard.
- Multiple emails can be analysed in one session; a history sidebar on the right lists past analyses and allows the user to reload any of them.
- The **aggregated sentiment chart** at the top of the page updates dynamically as new emails are analysed.

B. Creative Copilot

- Full-page chat interface resembling a modern AI assistant.
- User input box at the bottom with a send button; conversation history scrolls up automatically.
- Every AI message streams word-by-word (300ms delay per word).
- Next to the input area, provide **tone sliders** (professional ↔ casual, factual ↔ creative) that influence the canned responses (different mocked outputs based on slider values).
- Include **“Regenerate”** and **“Copy”** buttons below each AI message.
- Show a **“thinking”** animation (three pulsing dots) before the AI starts typing.
- Display a **token usage counter** that increments after each exchange.

C. Vision Board

- Prominent drag-and-drop zone for image upload (JPEG, PNG, WebP).
- Once uploaded, display the original image on the left half and an **AI overlay** on the right:
 - A matplotlib heatmap with semi-transparent colours simulating model attention, perfectly aligned over the image.
 - Below the images, show **detected objects** as tags with confidence bars.
 - A **generated caption** that appears letter-by-letter.

- Provide buttons to **download** the heatmap composite and the caption text.
 - Allow **multiple uploads**; show a gallery strip at the bottom to switch between analysed images.
-

4. Front-End Design Constraints

- The design must clearly communicate the **AI nature** of the product: use subtle gradients, glowing accents on AI-generated content, micro-interactions, and state indicators (e.g., "AI is analysing...").
 - Use a **coherent colour palette** that can be toggled between light and dark modes.
 - Typography: use at least two different font weights/styles to distinguish user-input from AI-output.
 - Incorporate **empty states** (e.g., "Upload an image to get started") and **error states** (e.g., "File too large") with helpful illustrations using emojis or simple Canvas drawings.
 - The entire app must fit within a single browser window without horizontal scrolling, even at 1024px width.
-

5. Provided Scaffolding

You are given a file `ai_stubs.py` with placeholder functions:

```
python
```

```
def summarize_email(text: str) -> dict:
    # Returns dict with keys: subject, sender, date, summary, sentiment, urgency, suggested_reply
    ...

def chat_response(message: str, tone_professional: float, tone_creative: float) -> str:
    # Returns a static multi-line response appropriate to tone sliders
    ...

def analyze_image(image_bytes: bytes) -> dict:
    # Returns dict with caption, objects [{"name":..., "confidence":...}], heatmap_data (2D array)
    ...
```

You may modify these stubs only to add delays (e.g., `time.sleep`) to simulate realistic inference times. No actual AI model may be used.

6. Deliverables

1. Complete Streamlit application (`app.py`, `pages/`, `utils/`, `components/` as needed) that runs with `streamlit run app.py`.
 2. A `requirements.txt` listing only `streamlit`, `matplotlib`, `numpy`, and `Pillow`.
 3. A 1-page PDF design rationale explaining:
 - How the interface guides the user through an AI workflow.
 - Key UX decisions for handling latency, confidence, and feedback.
 - How the visual design reinforces trust in AI outputs.
-

7. Evaluation Criteria

Criterion	Weight
UI/UX quality (layout, visual hierarchy, micro-interactions)	30%
AI integration feel (streaming, loading states, confidence visualisation)	25%
Code architecture (component reuse, session state, readability)	20%
Responsiveness & accessibility	15%
Creativity & innovation (extra thoughtful touches)	10%

8. Getting Started (Hints)

- Use `st.empty()` placeholders for streaming text.
 - Combine `st.columns` and `st.container` to build complex grid layouts.
 - Leverage `st.session_state` to persist chat history, analysed emails, and uploaded images.
 - For the heatmap, generate a matplotlib figure with `imshow` and a colormap, set `alpha` on the overlaid image, then display with `st.pyplot`.
 - Implement toast notifications using `st.toast`, `st.error`, `st.warning`, `st.success`.
-

Bring the AI experience to life through design – the intelligence is mocked, but the user's delight must be real.